

Module 1 — Integration Task: integration-1-dev-environment

Task type: Integration Task (rubric-graded by TA) **Submission:** GitHub PR URL → TalentLMS
Assignment (UIN 7): Branch: `integration/collab-setup` **Points:** 35

Overview

The lab gives your team a working environment. Now make it a working project — one that a new teammate, a TA, or a CLI agent can pick up and run in under two minutes without asking you a single question.

You will add the documentation, automation, and governance that turn a personal venv setup into a collaborative engineering artifact: a complete `README`, a comprehensive `.gitignore`, an idempotent `setup.sh`, an `AGENTS.md` contribution policy, and a `CHANGELOG.md`. The final deliverable is a PR whose description communicates the changes clearly enough that a real reviewer could verify them without talking to you.

By completing this task, you will:

- Write a professional `README` that any new contributor can follow to get started
- Author an `AGENTS.md` governance file that defines rules for AI-assisted contributions
- Create an idempotent `setup.sh` that automates environment setup
- Expand `.gitignore` to cover all project data and artifact patterns
- Document the initial project state in a `CHANGELOG.md`
- Write a structured PR description that serves as a communication artifact, not just a form

Context

The hospital admission data project is about to onboard a second team member. They clone your repo, look at the `README`, and have to ask: "How do I set this up? What data do I need? What's in each folder?" Every question they have to ask is a failure of your documentation.

This integration task produces the version of the repo that makes those questions unnecessary.

Note on AI tools: `AGENTS.md` is a written policy document — you author it the same way you author a `README`, using any text editor. No AI tool, paid or free, is required to complete this task. The 80% of learners without paid AI access complete this task identically to those who have it.

Prior Work You'll Use

This task builds directly on your Lab 1 submission. Continue working in the same `aispire-foundations` repo:

- Lab 1:** Your `requirements.txt`, `.gitignore` (with `.venv/`), and `test_environment.py` are already in place.

Create a new branch from `main` (or from your merged lab branch if it's already in `main`):

```
git checkout main
git pull origin main
git checkout -b integration/collab-setup
```

Deliverables

You must include all five of the following in your PR:

Deliverable 1 — Complete `README.md`

Rewrite `README.md` to include all seven required sections. Replace every `TODO` marker with real, project-specific content. A proficient `README` has no placeholder text remaining.

Required sections:

- Project Title** The name of the project. Example: `Hospital Admission Records Analysis`
- Team Members** Your name plus two fictional teammates. Include names only (or names and roles if you like).
- Project Overview** 2-3 sentences: what problem is being solved, for whom, and what the output will be. Write this as if explaining to a teammate joining the project, not a professor.
- Data Sources** Describe what data the project uses. Since data is not committed to the repo, include a note:

Data is not tracked in this repository. See the setup instructions below for how to obtain and place the data files before running any analysis.

Add a placeholder for where the data will go (e.g., `data/raw/admissions.csv`).

5. Setup Instructions Exact commands a new contributor should run, step by step:

```
git clone <repo-url>
cd <repo-name>
python -m venv .venv
```

Then the activation command — include all platform variants (see the table in the reading, Section 2.1):

```
# Mac / Linux
source .venv/bin/activate

# Windows Git Bash
source .venv/Scripts/activate

# Windows CMD
.venv\Scripts\activate.bat

# Windows PowerShell
.venv\Scripts\Activate.ps1
```

Then:

```
pip install -r requirements.txt
python test_environment.py
```

`test_environment.py` should print `Environment OK` when the setup is correct.

6. Project Structure A directory tree explaining each folder:

```
project-name/
├── README.md           - Project overview and setup instructions
├── CHANGELOG.md        - Record of notable changes
├── AGENTS.md           - AI contribution policy
├── requirements.txt     - Python dependencies
├── setup.sh            - Automated environment setup script
├── test_environment.py  - Environment validation
├── .gitignore          - Files excluded from version control
├── src/                - Production source code (importable modules)
├── notebooks/          - Exploratory analysis notebooks
├── data/               - Data directory (contents not committed to Git)
├── raw/                - Original unmodified data files
└── tests/              - Automated tests
```

Adjust the tree to reflect what's actually in your repo.

7. Contributing Branch naming convention (e.g., `feature/`, `setup/`, `fix/`), PR process summary, commit message format reminder.

Deliverable 2 — Expanded `.gitignore`

Expand `.gitignore` to cover all nine required patterns. Add them below the existing entries:

```
# Secrets and credentials
.env
*.key
*.pem

# Data files (never commit raw data to Git)
*.csv
*.parquet
*.xlsx
data/raw/

# OS artifacts
.DS_Store
Thumbs.db
```

The nine required patterns are: `.env`, `.venv/`, `*.csv`, `*.parquet`, `*.xlsx`, `__pycache__/`, `.ipynb_checkpoints/`, `data/raw/`, `.DS_Store`.

`.venv/` and `__pycache__/` should already be there from the lab. Add the rest.

Why these matter:

- `.env` must never be committed — it contains secrets
- `*.csv` and similar data file patterns prevent accidentally committing large data files
- `.DS_Store` is a macOS metadata file; `.ipynb_checkpoints/` is created automatically by Jupyter — neither belongs in version control

Deliverable 3 — `setup.sh`

Create a file named `setup.sh` in the repo root with the following contents:

```
#!/usr/bin/env bash
set -euo pipefail

python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python test_environment.py
echo "Setup complete."
```

After creating the file, make it executable:

```
chmod +x setup.sh
```

Then test it locally by running:

```
./setup.sh
```

It should complete without errors and print `Setup complete.`

What `set -euo pipefail` means:

- `-e`: exit immediately if any command fails (no silent errors)
- `-u`: treat unset variables as errors
- `-o pipefail`: propagate failures through pipes

This makes the script fail loudly and clearly instead of silently continuing after a broken step.

On Windows: `setup.sh` won't run natively in CMD or PowerShell. Run it in Git Bash, or execute the commands manually in sequence. Note this in your `README`'s Contributing section.

The PR checklist requires you to confirm `setup.sh` is executable and was tested locally before submission.

Deliverable 4 — `AGENTS.md`

Create a file named `AGENTS.md` in the repo root. It must include four sections:

1. Testing Requirements What must pass before any AI-assisted change is committed?

```
## Testing Requirements

All changes must pass 'python test_environment.py' before committing.
Any code added to 'src/' must have a corresponding test in 'tests/'
that passes with 'pytest tests/ -v'.
```

2. Secrets Policy What should never appear in a prompt or be committed?

```
## Secrets Policy

Do not include API keys, database passwords, file paths containing
personal data, or raw data content in any prompt. Never commit
.env, *.key, or any file containing credentials.
```

3. Scope Boundaries Which files may the agent edit? Which require human review?

```
## Scope Boundaries

Agents may edit files in 'src/' and 'notebooks/'.
Do not modify 'requirements.txt' without human review.
Do not modify 'setup.sh' without running and testing the result locally.
Do not touch '.gitignore' without confirming the change doesn't
accidentally exclude source files.
```

4. Reproducibility Standard What does "done" mean for an AI-assisted change?

```
## Reproducibility Standard

All AI-assisted changes require local-first execution: the change
must run locally and produce the expected output before it is
committed or pushed. "The AI generated it" is not a substitute
for running it.
```

Make the rules specific to your project. Replace or extend the examples above with language that reflects the actual structure of your repo (e.g., if your `src/` directory doesn't exist yet, describe what it will hold). Generic template text that hasn't been customized is a "Developing" score on the rubric.

Deliverable 5 — `CHANGELOG.md`

Create a file named `CHANGELOG.md` in the repo root documenting the initial project setup. Use the format below. Replace `[YYYY-MM-DD]` with the actual date you created the repo:

```
# CHANGELOG

## [YYYY-MM-DD] - Initial project setup

### Notes
- Project scaffold: README.md, .gitignore, requirements.txt, setup.sh
- AGENTS.md; AI contribution policy for this repository
- test_environment.py; environment validation script
- CHANGELOG.md; this file

### Notes
- Environment: Python 3.11, venv, pandas + matplotlib
- Repository created for hospital admission records analysis project
```

Write real notes — use the actual Python version you're running and the actual project purpose. The date should reflect when you actually set this up, not a placeholder.

Step-by-Step Guide

Step 1 — Set Up Your Branch

```
git checkout main
git pull origin main
git checkout -b integration/collab-setup
```

Confirm you're on the right branch: `git branch` should show `* integration/collab-setup`.

Step 2 — Build the Deliverables in Order

Recommended sequence:

- Write `CHANGELOG.md` first — it's the simplest and forces you to articulate what you're building
- Write `setup.sh`, make it executable, and test it locally before committing
- Expand `.gitignore` — run `git status` after to confirm data patterns are excluded
- Write `AGENTS.md` — make the rules specific to this project
- Rewrite `README.md` last — by now you know what's in the repo and can write accurate setup instructions

Step 3 — Commit Incrementally

Don't batch everything into one commit. Use meaningful commits:

```
git add CHANGELOG.md
git commit -m "Add CHANGELOG with initial project setup entry"

git add setup.sh
git commit -m "Add idempotent setup.sh for environment initialization"

git add .gitignore
git commit -m "Expand .gitignore to cover data files, secrets, and OS artifacts"

git add AGENTS.md
git commit -m "Add AGENTS.md: AI contribution policy"

git add README.md
git commit -m "Complete README with all required sections"
```

Step 4 — Push and Open the PR

```
git push --set-upstream origin integration/collab-setup
```

Open a PR on GitHub from `integration/collab-setup` to `main`. Use the PR description template below — fill it in completely.

PR Description Template

Copy this template into your PR description and complete every section. Incomplete PR descriptions are graded as "Developing" or "Beginning" on the PR quality dimension.

```
## What changed
- README.md: completed all 7 required sections (replaced all TODO: markers)
- .gitignore: expanded to cover .env, *.csv, *.parquet, *.xlsx, data/raw/, .DS_Store
- setup.sh: added idempotent environment setup script
- AGENTS.md: added AI contribution policy with 4 sections
- CHANGELOG.md: added initial project setup entry

## Why
- README: a new team member was unable to set up the project without asking quest
- .gitignore: data files and secrets must be excluded before any data work begins
- setup.sh: one-command setup removes ambiguity for new contributors and CI
- AGENTS.md: establishes explicit rules for AI-assisted contributions
- CHANGELOG.md: establishes the habit of recording what changed and when

## How to test
Clone the branch and run:
cd <repo-name>
git checkout integration/collab-setup
./setup.sh
Expected output: "Setup complete." with no errors.

## Checklist
- [ ] setup.sh is executable (chmod +x applied locally)
- [ ] setup.sh tested locally (ran ./setup.sh without errors)
- [ ] No sensitive data or large files committed
- [ ] .venv/ is in .gitignore
- [ ] .gitignore covers all 9 required patterns
- [ ] AGENTS.md rules are project-specific (not just template text)
- [ ] README has no remaining TODO: or [FILL IN:] placeholders

## Peer Validation
- Issue I filed on my partner's repo: [paste URL]
```

Grading

This task is rubric-graded by a TA. Your submission is evaluated on five dimensions:

Dimension	Weight	What earns a Proficient score
README completeness & quality + CHANGELOG.md	35%	All 7 README sections present with project-specific content (no placeholder text); setup instructions valid and copy-pasteable; CHANGELOG.md present with a valid dated entry
.gitignore coverage	15%	All 9 required patterns present including <code>.venv/</code> ; no learner source code accidentally excluded
setup.sh correctness	20%	Present, executable (<code>chmod +x</code> applied), runs without error, uses venv commands, includes <code>set -euo pipefail</code>
AGENTS.md quality	15%	All 4 required sections present; rules are specific to this project (not just the template text); scope boundaries reference actual project files
PR description quality	15%	Structured: what changed + why + how to test; written for a real reviewer, not a grader

Full rubric details are available from your TA after the submission.

A note on "proficient": Proficient means the work does what it's supposed to do without errors, and the content is specific to your project. Excellent means you went beyond the minimum — more complete .gitignore patterns, more detailed AGENTS.md rules, README that genuinely communicates rather than just checks boxes. Aim for work you'd be comfortable showing in a portfolio.

Submitting

- Confirm all deliverables are committed to `integration/collab-setup`
- Confirm the PR description is fully completed using the template above
- Filed a peer validation issue on your partner's repo (with What worked / What was confusing / One suggestion)
- Added the issue link to your PR description under **Peer validation**
- Open a Pull Request from `integration/collab-setup` to `main` (if not already open)
- Copy your PR URL into TalentLMS → **Module 1 — Integration 1** to submit this assignment
- The PR must be open (not merged) at submission time. Do not merge your own PR.

Tips

- Run `./setup.sh` on a fresh terminal (no venv active) to confirm it works end-to-end before submitting.
- After writing `AGENTS.md`, read it aloud as if you were a new team member seeing the repo for the first time. Does it answer the questions you'd actually have?
- The `README "Setup Instructions"` section is the most commonly incomplete part. Include all platform activation variants — not just the one you use.
- If `git status` shows `.venv/` is on tracked files list, `.gitignore` is not working. Open `.gitignore` and confirm `.venv/` is on its own line with no leading spaces.

Peer Validation

Your integration task prepared your repo so a teammate can test it and get to work. Before you submit, find out if it actually works — pair up with a classmate and done each other's repos.

Steps

- Find a partner who has also completed their integration deliverables and opened their PR
- Clone your partner's `aispire-foundations` repo
- Check out their `integration/collab-setup` branch
- Run their `setup.sh` — does it complete without errors?
- Read their `README` — could you set up the project without asking them anything?
- Review their `AGENTS.md` — are the rules specific to their project or generic template text?
- Open a **GitHub Issue** on your partner's repo titled `Peer Validation - [Your Name]` with three sections:
 - What worked:** (setup.sh ran clean, README was clear, etc.)
 - What was confusing or broken:** (setup.sh failed on line X, README missing activation command for Windows, etc.)
 - One suggestion:** (a specific improvement, not "looks good")
- Update the link to the issue you filed (on your partner's repo) to your own PR description — see the updated PR template below